

Numerieke benadering

Hoofdstuk 2: Benadering van vectoren

Bespreek het begrip projector in de ruimte $\mathbb{C}^m$

- (a) Geef definitie. Waarop wordt geprojecteerd? In welke richting?
- (b) Leid een expliciete uitdrukking af voor projector P_{S_1, S_2} die projecteert op $S_1 = \langle a_1, \dots, a_n \rangle$ in de richting van $S_2 = \langle b_1, \dots, b_n \rangle^\perp$.
- (c) Bewijs dat een projector P een orthogonale projector is indien $P = P^*$.
- (d) Beschouw de uitmiddelingmatrix T , waarbij

$$T \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} = \frac{x_1 + x_2 + \dots + x_m}{m} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

Is T een projector? Een orthogonale projector?

Een **projector** is een matrix die **idempotent** is, dus $P = P^2$. Er wordt geprojecteerd op de kolomruimte van P in de richting van de nulruimte van P .

$$v = Pv + (I - P)v$$

De **complementaire projector** $\tilde{P} = I - P$ is ook een projector die projecteert op de nulruimte in de richting van de kolomruimte.

We projecteren een vector v op A . Stel $v_1 = Ay$, de projectie van v op S_1 . Dan eisen we dat $(v - v_1) \in S_2$ of $(v - v_1)^\perp \in \mathcal{R}(B)$. Dus $B^*(v - Ay) = 0$. Dus $B^*v = B^*Ay$.

Omdat B^*A inverteerbaar is, kunnen we stellen dat $v_1 = Ay = A(B^*A)^{-1}B^*v$.

De voorwaarde is nodig: Neem matrix Q met de eerste n kolommen een orthonormale basis voor de kolomruimte van P , en de laatste $m - n$ kolommen een voor de nulruimte van P .

$$PQ = [q_1 \dots q_n \ 0 \dots 0] \\ Q^*PQ = \text{diag}(1 \dots 1 \ 0 \dots 0)$$

Uit deze vorm kunnen we halen dat:

$$(Q^*PQ)^* = Q^*P^*Q$$

De voorwaarde is voldoende: Neem een willekeurige $x = Pu$ uit de kolomruimte van P en een y uit de nulruimte van P (dus $Py = 0$).

$$x^*y = u^*P^*y = u^*Py = 0$$

Hieruit volgt dat de kolomruimte en de nulruimte van P loodrecht op elkaar staan.

Om na te kijken of T een projector is, passen we de algemene definitie van een projector toe ($T^2 = T$). Om na te kijken of het een orthogonale projector is, proberen we best de structuur van T uit te schrijven. Op basis daarvan kunnen we beredeneren dat $T^* = T$.

Besprek hoe je met Householder-transformaties (reflecties) een QR-factorisatie van een matrix $A \in \mathbb{C}^{m \times n}$ berekent.

- Wat is de betekenis van de uitdrukking $F_m = I_m - 2 \frac{vv^*}{v^*v}$, met $v = x + \text{sign}(x_1) \|x\|_2 e_1$? Leid deze af, vertrekkende van projector $P_{\langle v \rangle^\perp} = I_m - vv^*$.
- Waarom wordt rekening gehouden met het teken van x_1 ?
- Leg stap voor stap uit hoe je matrix R berekent, alsook matrix Q . Beschrijf het algoritme in pseudocode.
- Geef de pseudocode om de Hessenbergmatrix te genereren met Householder transformaties.
- Wat is de kost in functie van enkel m ?
- Kan je bovenstaande methode ook gebruiken om een matrix om te vormen tot een bovendriehoeksmatrix?
- Geef het verband met eigenwaardenbenadering

De bedoeling van een **Householdertransformatie** is om in een bepaalde kolom de laatste $m - 1$ rijelementen nul te maken.

Een reflector is een **spiegeling** ten opzichte van het orthogonaal complement van een vector v , beeldt een vector x af op $R_{\langle v \rangle^\perp} x = x + 2(P_{\langle v \rangle^\perp} x - x)$. Hieruit volgt:

$$R_{\langle v \rangle^\perp} = I - 2 \frac{vv^*}{v^*v}$$

We kunnen zo'n reflector dan gebruiken om nullen te creëren in een bepaalde kolom. Het eindresultaat moet voldoen aan:

$$R_{\langle v \rangle^\perp} x = \pm \|x\| e_1$$

De richting v van deze reflectie is die van $x - R_{\langle v \rangle^\perp} x$.

Hieruit leiden we af dat $v = x \mp \|x\| e_1$.

Het teken van het resultaat is in principe vrij te kiezen. Omwille van numerieke stabiliteit willen we echter **gevaarlijke aftrekkingen** vermijden. We kiezen dus het teken zodat $\|v\|$ maximaal wordt, vandaar $\text{sign}(x_1)$.

Het algoritme gaat niet telkens de Householdertransformaties apart opstellen. We kunnen de volgende operatie nl. snel uitrekenen:

$$F_m q = q - 2 \frac{v^* q}{v^* v} v$$

Algoritme:

- Voor elke kolom i , creëer de Householdertransformatie:

$$Q_{i,m-i+1}^* = \begin{bmatrix} I_{i-1} & 0 \\ 0 & F_{m-i+1} \end{bmatrix}.$$

- Als je die Householdertransformaties achtereenvolgend uitvoert op A , dan bekom je R . De ketting van transformaties vormt Q^* .

$$A = \begin{bmatrix} x & x & x \\ x & x & x \\ x & x & x \\ x & x & x \end{bmatrix} \xrightarrow{Q_{1,4}^*} \begin{bmatrix} \mathbf{x} & \mathbf{x} & \mathbf{x} \\ 0 & \mathbf{x} & \mathbf{x} \\ 0 & \mathbf{x} & \mathbf{x} \\ 0 & \mathbf{x} & \mathbf{x} \end{bmatrix} \xrightarrow{Q_{2,4}^*} \begin{bmatrix} x & x & x \\ 0 & \mathbf{x} & \mathbf{x} \\ 0 & 0 & \mathbf{x} \\ 0 & 0 & \mathbf{x} \end{bmatrix} \xrightarrow{Q_{3,4}^*} \begin{bmatrix} x & x & x \\ 0 & x & x \\ 0 & 0 & \mathbf{x} \\ 0 & 0 & 0 \end{bmatrix}.$$

Het algoritme in het HB (p. 45) is specifieker:

- Het ‘uitvoeren’ van de transformaties wordt rechtstreeks in het algoritme opgenomen: het resultaat is dat de diagonaalelementen van R telkens veranderen en er nullen worden gecreëerd daaronder.
- De kolommen rechts van de huidige kolom i veranderen ook (zie figuur hierboven).
- Op die manier moet je nooit echt een matrix-matrixproduct uitrekenen.

Pseudocode om matrix om te zetten in **Hessenbergvorm** via Householdertransformaties:

Algoritme 8 Impliciet verschoven QR-algoritme - gelijkvormige Hessenberg-matrix via Householder-transformaties

```

1:  $H = A$ 
2: for  $j = 1$  to  $m - 2$  do
3:    $x = H_{j+1:m,j}$ 
4:    $v_j = x + \text{sign}(x_1) \|x\|_2 \mathbf{e}_1$ 
5:    $v_j = v_j / \|v_j\|_2$ 
6:    $H_{j+1,j} = -\text{sign}(x_1) \|x\|_2$ ,  $H_{j+2:m,j} = 0$ 
7:   for  $k = j + 1$  to  $m$  do
8:      $H_{j+1:m,k} = H_{j+1:m,k} - 2 (v_j^* H_{j+1:m,k}) v_j$ 
9:   end for
10:  for  $k = 1$  to  $m$  do
11:     $H_{k,j+1:m} = H_{k,j+1:m} - 2 (H_{k,j+1:m} v_j) v_j^*$ 
12:  end for
13: end for
14: for  $i = 1$  to  $m$  do
15:    $w = \mathbf{e}_i$ 
16:   for  $k = m - 2$  downto  $1$  do
17:      $w_{k+1:m} = w_{k+1:m} - 2 (v_k^* w_{k+1:m}) v_k$ 
18:   end for
19:    $Q_{1:m,i} = w$ 
20: end for
```

Rekenkost: $O(m^3)$, de matrix is vierkant dus $m = n$.

Je kan Householdertransformaties uiteraard ook gebruiken om een matrix om te zetten tot een **bovendriehoeksmatrix**. Een aantal aanpassingen zijn nodig:

- Nu zetten we uiteraard alle elementen onder de diagonaal op 0, i.p.v. enkel de elementen onder de onderdiagonaal.
- Enkel langs links de Householdertransformatie uitvoeren.

Het omzetten naar Hessenbergvorm dient als een soort van ‘preprocessing’ stap bij **het impliciet verschoven QR-algoritme** om eigenwaarden van een matrix te vinden. Voor elke iteratiestap bepalen we een nieuwe matrix $A^{(k)}$ vanuit de Hessenbergmatrix $A^{(k-1)}$. Die nieuwe matrix $A^{(k)}$ heeft de volgende eigenschappen:

- Gelijkvormig met $A^{(k-1)}$ en heeft dus dezelfde eigenwaarden
- Ook in Hessenbergvorm
- Eerste kolom van $Q^{(k)}$ is gelijk aan $p^{(k)}(A^{(k-1)})e_1$

Deze 3 eigenschappen leiden ertoe dat we impliciet een deelruimte-iteratie toepassen met extra flexibiliteit ingebouwd omdat we verschuiving uitvoeren. De Hessenbergvorm laat toe om recursief eigenwaarden af te splitsen. Het afsplitsen zelf is mogelijk omdat de methode equivalent is met §9.2.2.1. Dit is een ruwe schets van de methode, voor meer informatie zie de vragen bij hoofdstuk 9.

Beschouw het overgedetermineerde stelsel $Ax \approx b$, waarbij $A \in \mathbb{R}^{m \times n}$, met $m > n$.

- Leg uit hoe je met behulp van een *QR*-factorisatie de kleinste-kwadratenoplossing kan bepalen.
- Heb je voor het berekenen van de oplossing de onvolledige of volledige *QR*-factorisatie nodig? Verklaar!
- Stel A is van dimensie $10^7 \times 10$. Welk algoritme zou je voor de *QR*-factorisatie gebruiken? Waarom?
- Waarom is deze aanpak beter dan rechtstreeks het normaalstelsel op te lossen?

Zie §2.4.2 voor uitleg hoe je de **QR-factorisatie** kan gebruiken om de kleinste-kwadratenoplossing te bepalen. Op het examen wil je waarschijnlijk ook nog uitleggen waar de *QR*-factorisatie voor staat (orthonormale geneste basis creëren, R zijn de coëfficiënten bij de lineaire combinaties in de nieuwe basis).

Je hebt hier niet de volledige *QR*-factorisatie nodig, omdat we bij het berekenen van b_1 enkel de eerste n kolommen van Q nodig hebben. Het stelsel zelf gebruikt ook de onvolledige $\hat{R}$:

$$\hat{R}x = b_1$$

We maken de trade-off tussen **orthogonaliteit** en **rekenkost**. Het is van het begin al duidelijk dat Givensrotaties niet efficiënt zijn: we moeten voor elk element onder de diagonaal apart een Givensrotatie gaan opstellen, wat gigantisch inefficiënt is.

	<i>Gram-Schmidt</i>	<i>Householder</i>
<i>Rekenkost</i>	$\sim 2mn^2$	$4m^2n - \frac{2}{3}n^3$ want we hebben zowel Q als R nodig
<i>Orthogonaliteit</i>	Gevaarlijke aftrekkingen zorgen voor orthogonaliteitsfouten!	Unitaire transformaties, dus heel goed

Rekenkost lijkt me hier wel de doorslaggevende factor. De kracht van *Gram-Schmidt* is dat het kolom per kolom gaat werken. Er zijn hier slechts 10 kolommen (die wel groot zijn, maar toch), waardoor het algoritme niet super veel stappen moet doorlopen. Het feit dat we zowel Q als R moeten opstellen, zorgt ervoor dat *Householder*transformaties veel meer rekenkost vragen: de Q -matrix wordt opgesteld via rij-operaties, en er zijn gigantisch veel rijen.

Om de orthogonaliteit van *Gram-Schmidt* te verbeteren, kunnen we gebruik maken van het gewijzigde algoritme + **herorthogonalisatie** (stapsgewijs of simultaan). Op deze manier wordt het ‘nadeel’ van een grotere orthogonaliteitsfout weggewerkt.

Rechtstreeks het normaalstelsel oplossen is een slecht geconditioneerd probleem: conditiegetal is $\kappa(A^*A) = \kappa(A)^2$, dus afrondingsfouten hebben een groot effect op het eindresultaat. Door met unitaire transformaties te werken, $\kappa(U) = 1$, is er in theorie dus geen effect van perturbaties op de input te merken in het eindresultaat. Unitaire transformaties zorgen dus voor (achterwaarts) stabiele algoritmes (op voorwaarde dat de unitaire matrices ook op een achterwaarts stabiele manier worden bepaald).

Bespreek de QR-factorisatie en hoe je die kan opstellen.

- (a) Bespreek de geneste structuur van de QR-factorisatie en waarom deze structuur kan gebruikt worden in de Arnoldi-iteratie.
- (b) Bespreek de meetkundige interpretatie van Gram-Schmidt orthogonalisatie.
- (c) Is het GS-algoritme stabiel? Geef enkele simpele methodes waarmee we de stabiliteit van het GS-algoritme kunnen verbeteren.
- (d) Bespreek kort hoe men met Givensrotaties een QR-factorisatie kan opstellen.
- (e) Bespreek kort de meetkundige interpretatie van Householder transformaties en hoe we hiermee een QR-factorisatie kunnen opstellen.
- (f) Vergelijk de drie methodes op basis van volgende eigenschappen: stabiliteit, orthogonaliteit en rekenkost. Bij welk type matrices verkies je elke methode?

De eerste 5 deelvragen zijn grotendeels al beantwoord geweest en kunnen gemakkelijk opgezocht worden in de cursus.

	<i>Gram-Schmidt</i>	<i>Givensrotatie</i>	<i>Householder</i>
<i>Rekenkost</i>	$\sim 2mn^2$	Enkel R: $3mn^2 - n^3$ $6m^2n - n^3$	Enkel R: $2mn^2 - \frac{2}{3}n^3$ $4m^2n - \frac{2}{3}n^3$
<i>Orthogonaliteit</i>	Gevaarlijke aftrekkingen zorgen voor orthogonaliteitsfouten!	Unitaire transformaties, dus machineprecisie	Unitaire transformaties
<i>Stabiliteit</i>	Slechter	Zeer goed	Zeer goed
<i>Type matrices</i>	Kleine tot middelgrote, goed geconditioneerde matrices, problemen waar Q expliciet nodig is, problemen waar on-volledige QR-factorisatie voldoende is	Spaarse matrices, toevoegen van rijen en kolommen in bestaande QR	Grote, dense, slecht geconditioneerde matrices waar precisie belangrijk is en Q niet expliciet nodig is

Hoofdstuk 3: Benaderen in een unitaire ruimte

Beschouw de functionaal $I(f) = \max_{x \in [a, b]} |f(x)|$.

- (a) Bewijs dat $I(f)$ een norm definieert op de ruimte $C[a, b]$ (ruimte van continue functies op het interval $[a, b]$).
- (b) Is het een strikte norm? Argumenteer.
- (c) Voldoet deze norm aan de zogenaamde parallelogram-gelijkheid? Welk gevolg kan je daaruit trekken?

1. $I(f) > 0$: triviaal

2. $I(f) = 0$ als en alleen als $f(x) = 0$

Triviaal, aangezien alles binnenin het 'max'-gedeelte positief is. Daardoor kan het maximum van $|f(x)|$ enkel 0 zijn als $f(x) = 0$ in elk punt van het interval.

3. $I(af) = |a|I(f)$:

Triviaal, aangezien $|af(x)| = |a| \cdot |f(x)|$

De absolute waarde van een functie beschrijft ook een norm.

4. $I(f + g) \leq I(f) + I(g)$: beschouw $|f(x) + g(x)| \leq |f(x)| + |g(x)|$

De functionaal beschrijft dus een norm.

Neem nu $[a, b] = [0, 1]$, $f(x) = 1$ en $g(x) = x$. Er geldt dus dat $f(x) \neq g(x)$ en $I(f) = I(g) = 1$. Dan geldt: $I(f + g) = 2$. Dit betekent dus dat de eenheidsbol volgens deze norm niet strikt convex is en bijgevolg is de functionaal dus geen strikte norm.

De parallelogramgelijkheid stelt dat:

$$I(f + g)^2 + I(f - g)^2 = 2(I(f)^2 + I(g)^2)$$

Neem nu $[a, b] = [0, 1]$, $f(x) = 1$ en $g(x) = x$. Als we de parallelogramgelijkheid uitrekenen zien we dat dit niet overeenkomt. Hieruit kunnen we afleiden dat het geïnduceerd scalair product $I(f) = \sqrt{(f, f)}$ niet bestaat. Dit volgt trouwens ook uit het feit dat het geen strikte norm is.

Beschouw een unitaire ruimte, uitgerust met scalair product $(\cdot, \cdot)$.

- (a) Bespreek in detail het Gram-Schmidt algoritme voor het opstellen van een geneste orthonormale basis van een deelruimte, vertrekkende van een niet-orthogonale basis. Leg daarbij de relatie met orthogonale projecties.
- (b) Wat kan je zeggen over de hoeveelheid rekenwerk, in termen van het aantal benodigde scalaire producten?
- (c) Wat is herorthogonalisatie in deze context? Wanneer/ waarom kan dit belangrijk zijn?
- (d) Stel dat je een orthogonale rij veeltermen wil opstellen met betrekking tot een bepaald scalair product. Is het toepassen van het Gram-Schmidt algoritme vertrekkende van $\{1, x, x^2, \dots\}$ de computationeel meest efficiënte methode? Waarom wel/niet?
- (e) Kunnen we andere methodes steunend op unitaire transformaties gebruiken om te orthogonaliseren? Waarom wel/niet?

Het **Gram-Schmidt algoritme** gaat een geneste orthonormale basis opstellen voor een ruimte A. Het verandert de basisvectoren van A dus naar orthonormale vectoren in de unitaire matrix Q. De coëfficiënten van de vector van A in basis Q worden opgeslagen in matrix R, zo bekomen we de QR factorisatie.

Het algoritme werkt als volgt:

- Vertrek van een basisvector a_j .
- Trek alle bijdragen van de vorige orthonormale vectoren één voor één af van de huidige basisvector. Op die manier projecteren we a_j orthogonaal op $\langle q_1 \dots q_{j-1} \rangle$.
- Normaliseer de nieuwe basisvector.

De meetkundige interpretatie is dus dat we alle vectoren projecteren op de al bestaande basisvectoren. De vector die dan overblijft is dan de 'restvector'. Die staat loodrecht op de vorige vectoren, aangezien we een **orthogonale projectie** gebruiken.

Om **gevaarlijke aftrekkingen** te vermijden, kunnen we het **gewijzigde GS-algoritme** gebruiken. De meetkundige interpretatie is dat we de vectoren steeds loodrecht projecteren op één andere basisvector. Wiskundig is dit equivalent met te projecteren op alle vorige basisvectoren tegelijk.

Rekenwerk: n stappen waarin telkens een lus zit van j-1 scalaire producten, dus $O(n^2)$ aantal scalaire producten. Elke iteratie wordt ook een norm berekend om te normaliseren, dus $O(n^2 + n) = O(n^2)$.

Gram-Schmidt orthogonalisatie is niet numeriek stabiel. Bij toenemende n gaat de orthogonaliteit van bijkomende vectoren snel verloren, zeker wanneer een nieuwe vector bijna lineair afhankelijk is van de vorige. Zoals eerder besproken kunnen we gevaarlijke aftrekkingen vermijden, maar om orthogonaliteit tot machineprecisie te hebben maken we gebruik van **herorthogonalisatie**:

- **Stapsgewijs**: tijdens elke iteratie orthogonaliseren we gewoon opnieuw, maar nu met de orthonormale vector q_j die we in het eerste deel van de iteratie berekende, als inputvector
- **Simultaan**: na de volledige orthogonalisatie, voeren we die gewoon opnieuw uit.

Bij het gewijzigde algoritme is het vaak voldoende om één keer te herorthogonaliseren. Op die manier hebben we praktisch perfect orthogonale basisvectoren, die belangrijk zijn om de numerieke stabiliteit van andere algoritmes te vrijwaren (bv. oplossen normaalstelsel).

Voor orthogonaliseren van veeltermfuncties bestaat er een **drietermsrecursiebetrekking** (zie formularium en cursus voor bewijs). Deze recursiebetrekking moet maar $O(n)$ scalaire producten berekenen, omdat de resultaten uit de vorige stappen kan opslaan en hergebruiken later.

Unitaire transformaties vormen de matrix A om naar R, terwijl Gram-Schmidt de matrix A omvormt tot Q. Dit laatste proces is dus gemakkelijk om algemeen te maken, aangezien Gram-Schmidt enkel kolomoperaties uitvoert = operaties op vectoren. Om Q te berekenen, moeten er unitaire transformaties op rijvectoren worden uitgevoerd, maar een vectorruimte veralgemeent niet zomaar tot kolomruimte én rijruimte, er is dus een compatibiliteitsprobleem.

Beschouw de functie $f(x, y) = \frac{|x-y|}{1+|x-y|}; x, y \in \mathbb{R}$.

- (a) Is f een afstand?
- (b) Kunnen we de geïnduceerde norm definiëren voor f ?
- (c) Geef een voorbeeld van een strikte norm. Waarom is deze norm strikt?
- (d) Bewijs dat de beste benadering in een strikt genormeerde ruimte bestaat en uniek is.

Om na te gaan dat $f(x, y)$ een **afstand** is, kijken we de vier voorwaarden van een afstand na. Dan zien we inderdaad dat f een afstand is.

We kunnen geen **geïnduceerde norm** definiëren aangezien f niet homogeen is. Het is wel **supermetrisch**, maar beide eisen moeten voldaan zijn om een norm te induceren.

Een norm is strikt als en slechts als voor alle $x_1, x_2 \in C$ met $x_1 \neq x_2$ en $\|x_1\| = \|x_2\| = 1$ geldt dat $\|x_1 + x_2\| < 2$. Een voorbeeld van een **strikte norm** is de **Euclidische norm**.

Voor het bewijs dat de benadering in een strikt genormeerde ruimte uniek is, zie stelling 3.2.9 op p. 66 in de cursus.

Hoofdstuk 4: Veeltermbenaderingen

Beschouw een orthogonale rij veeltermen $\{\phi_0(x), \phi_1(x), \phi_2(x), \dots\}$ met betrekking tot een scalair product van de vorm $(f, g) = \int_a^b w(x)f(x)g(x)dx$ met $w(x) > 0$ voor $x \in [a, b]$.

- (a) Wat kan je zeggen over het aantal nulpunten van ϕ_n en hun ligging? Een bewijs is niet nodig.
- (b) Bewijs dat de nulpunten van ϕ_n kunnen berekend worden als eigenwaarden van matrix

$$A = \begin{bmatrix} \alpha_1 & \mu_1 & & & & \\ \nu_1 & \alpha_2 & \mu_2 & & & \\ & \nu_2 & \alpha_3 & \mu_3 & & \\ & & \nu_3 & \alpha_4 & \mu_4 & \\ & & & \ddots & \ddots & \ddots \\ & & & & \nu_{n-2} & \alpha_{n-1} & \mu_{n-1} \\ & & & & & \nu_{n-1} & \alpha_n \end{bmatrix}.$$

Hint: de recursiebetrekking voor de orthogonale rij kan geschreven worden in de vorm

$$\nu_{k-1}\phi_{k-2}(x) + \alpha_k\phi_{k-1}(x) + \mu_k\phi_k(x) = x\phi_{k-1}(x).$$

- (c) Ik wil een gegeven functie benaderen door een veelterm van een bepaalde graad. Ik bepaal de coëfficiënten van de veelterm door middel van interpolatie in de nulpunten van een Chebyshev-veelterm. Is dit een goede aanpak? Geef uitleg.

De veelterm $\phi_n(x)$ die behoort tot een orthogonaal stel veeltermen over een interval $[a, b]$ heeft n enkelvoudige reële nulpunten in het open interval (a, b) .

Bewijs (niet gevraagd): Stel dat er maar $m < n$ reële nulpunten zijn. Beschouw dan:

$$\psi(x) = \phi_n(x)(x - x_1) \dots (x - x_m)$$

Telkens wanneer de functie van teken wisselt (in een nulpunt), wordt dit tegengewerkt door de extra factoren. Alle nulpunten worden dus verdubbeld. Dan stellen we dat:

$$\int_a^b w(x)\phi_n(x)(x - x_1) \dots (x - x_m)dx \neq 0$$

Maar een veelterm van graad n staat orthogonaal op alle veeltermen van een lagere graad. Door contradictie is de stelling dus bewezen.

We kunnen de recursiebetrekking herschrijven zoals in de opgave. Uiteindelijk bekomen we:

$$A \begin{bmatrix} \phi_0(x) \\ \vdots \\ \phi_{n-1}(x) \end{bmatrix} = x \begin{bmatrix} \phi_0(x) \\ \vdots \\ \phi_{n-1}(x) \end{bmatrix} - \mu_n \begin{bmatrix} 0 \\ \vdots \\ \phi_n(x) \end{bmatrix}$$

Als we hier een nulpunt van $\phi_n(x)$ invullen, dan bekomen we een eigenwaardeprobleem. De eigenwaarden zoeken van A geeft ons dus de nulpunten van deze functie.

Of een benadering met Chebyshev interpolatiepunten goed is, hangt van twee zaken af:

1. Wat verstaan we onder een 'goede' benadering?
 - **Kleinste-kwadratencriterium:** De Chebyshev basis is orthogonaal t.o.v. de gewichtsfunctie $w(x) = 1/\sqrt{1-x^2}$. Als we daarna de veeltermcoëfficiënten uitrekenen (stelsel met Grammatrix oplossen) t.o.v. een andere gewichtsfunctie

zal de Chebyshev basis een scheve basis zijn wat leidt tot numeriek instabieler benaderingen.

- **Minimax-criterium:** Als we op zoek zijn naar een goede benadering die de maximale fout minimaliseert, dan hebben we volgens de **equi-oscillatiestelling** $n+2$ extremaalpunten nodig die afwisselen van teken. De Chebyshevpunten voldoen benaderend aan die eis. Omdat het residu wordt benaderd door de volgende term in de drietermsrecursiebetrekking, zijn Chebyshevpunten een goede keuze om aan dit criterium te voldoen.

(Dit in combinatie met het feit dat er een snelle FFT-algoritmes bestaan om te schakelen tussen Chebyshevpunten en de coëfficiënten van de interpolerende veelterm maakt dit een goede keuze).

2. Welke punten willen we goed benaderen? Sommige functies zijn 'moeilijker' de benaderen in bepaalde punten (bv. de Rungefunctie is moeilijk te benaderen aan de rand van het interval door asymptotische eigenschappen). Het verdelen van de knooppunten speelt hier een rol. Chebyshevpunten zijn meer geconcentreerd aan de randen, waardoor daar een betere benadering is.

In het algemeen zijn Chebyshevpunten wel een goede keuze en worden ze in veel toepassingen gebruikt. Ze zijn over het algemeen beter dan andere basissen die we in de cursus hebben besproken zoals Legendre en monomiale veeltermen.

(Alternatieven als B-splines kunnen afhankelijk van de functie wel een betere stuksgewijze benadering opleveren.)

Hoofdstuk 5: Benaderen met splinefuncties

Bespreek het begrip kubische spline-functie.

- (a) Geef een definitie.
- (b) Wat is de dimensie van de bijhorende spline-ruimte ? Bewijs deze formule.
- (c) Definieer en teken een basis van afgeknotte machtsfuncties en een basis van B-splines.
- (d) Leg gedetailleerd uit hoe je die B-spline basis zou gebruiken voor het benaderen van een experimenteel opgemeten functie.
- (e) Illustreer met een voorbeeld hoe het resulterende lineaire stelsel er uit ziet (spaarsheidspatroon is voldoende, de waarde van de matrix-elementen is niet nodig).

Bespreek B-splines.

- (a) Geef de definitie van genormaliseerde B-splines en teken een basis van B-splines. Zorg dat op je tekening de belangrijkste eigenschappen van B-splines zichtbaar zijn.
- (b) Hoe worden deze B-splines gecombineerd tot de algemene splinefunctie $s(x)$?
- (c) Leg uit en bewijs hoe we deze splinefunctie efficiënt kunnen evalueren.
- (d) Wat verandert er als er samenvallende knooppunten gebruikt worden? Wanneer zouden we dit kunnen gebruiken?

Hoofdstuk 6: Discrete benadering

Beschouw een tweedimensioneel benaderingsprobleem: we zoeken een benadering van discrete meetdata $f(x_i, y_j) \approx f_{i,j}, 1 \leq i \leq M, 1 \leq j \leq N$, voorgesteld als een veelterm

$$z(x, y) = \sum_{k=0}^m \sum_{\ell=0}^n x^k y^{\ell}$$

met separabele gewichten $w_{i,j} = w_i^{(1)} w_j^{(2)}$.

- Hoe bekom je een orthogonale basis voor dit veeltermbenaderingsprobleem? Bewijs.
- Met een orthogonale basis reduceert het oplossen van het normaalstelsel zich tot het evalueren van de coëfficiënten van de beste benadering. Toon dit aan.
- Wat is een goede (efficiënte) manier om deze coëfficiënten te berekenen? Geef de formules en illustreer grafisch.
- Vergelijk de rekencomplexiteit met de directe methode.
- Stel dat $z(x, y)$ nu een splinefunctie was in twee veranderlijken met een genormaliseerde B-spline basis. Hoe zou je deze splinefunctie dan efficiënt evalueren? Maak de vergelijking met (c).

Voor discrete data wordt dit gedaan met de **methode van Forsythe**, dat je kan beschouwen als de **drietermsrecursiebetrekking** uit §4.2.1 in de cursus, maar dan voor scalaire producten. QR-factorisatie is uiteraard ook mogelijk, maar is duurder in rekentijd ($O(n^2)$ vs. $O(n)$ met n het aantal scalaire producten).

Bewijs: staat op p. 86 in het HB. Het scalair product is een veralgemening en moet hier dan aangepast worden naar het scalair product voor discrete 2D-functies.

- Geef de basisiteratie vanuit het formularium
- Schrijf $x\phi_{k-1}(x)$ op als een lineaire combinatie van basisvectoren $\phi_0(x)$ t.e.m. $\phi_k(x)$ en schrijf de coëfficiënten uit vanuit het formularium (scalair product / scalair product)
- Alle veeltermen van een hogere graad staan loodrecht op die van een lagere graad. Sommige scalaire producten worden dus 0, en er blijven maar 3 termen over.
- Dit herschrijven naar de vorm van de drietermsrecursiebetrekking geeft het resultaat.

De tweede deelvraag gaat terug naar het algemene stelsel van vergelijkingen op p. 74 met de **grammatrix**.

- Construeer de loodrechte projector P_D . De eis dat $\vec{v} - \vec{v}_1$ loodrecht staat op de projectie leidt tot een stelsel van vergelijkingen dat kan beschreven worden als $Gc = b$ met G de grammatrix.
- Als de veeltermen orthogonaal zijn reduceert dit stelsel zich. De uitdrukking is te vinden in het formularium.

Het vinden van de coëfficiënten voor de discrete 2D-benadering wordt beschreven op p. 154. Het basisconcept is **herhaalde eendimensionale benadering**. De uitwerking staat in het formularium.

- Eerst worden er N kleinste-kwadratenbenaderingen van graad m voor discrete functies van lengte M opgesteld evenwijdig met de x -as
- Daarna worden er $m+1$ kleinste-kwadratenbenaderingen van graad n voor discrete functies van lengte N opgesteld evenwijdig met de y -as
- Figuur op p. 155 geeft een grafische illustratie.

De rekentijd gaat dus van $(n + 1)(m + 1)NM$ naar $NM(m + 1) + (n + 1)(m + 1)N$.

Beschouw een discreet benaderingsprobleem: we zoeken een benadering van discrete meetdata $f(x_i) \approx f_i, 1 \leq i \leq N$ als een veelterm.

- (a) Als we een discrete kleinste-kwadratenbenadering opstellen, wat is dan het scalaire product waarvan we vertrekken?
- (b) Toon aan dat de oplossing van het normaalstelsel overeenkomt met die van een overgedetermineerd stelsel.
- (c) Bij benadering van discrete data is overfitting een moeilijk probleem. Bespreek kort hoe overfitting *gedetecteerd* kan worden.
- (d) Een manier om overfitting te vermijden is regularisatie. Geef een definitie van (en motivatie voor) Tikhonov regularisatie.
- (e) Toon aan dat Tikhonov regularisatie slechts een kleine aanpassing geeft aan het normaalstelsel voor de kleinste-kwadratenbenadering.

We gebruiken het **discreet scalair product**, zie formularium. De reden is dat de vectoren die orthogonaal zijn voor het continue scalaire product voor functies, dat vaak (net) niet zijn voor het discrete scalair product, waardoor we niet de vereenvoudigde manier kunnen hanteren om de coëfficiënten uit te rekenen.

We hebben de eis dat $y_n(x_i) = f_i$ voor alle N meetpunten. Dit leidt tot een **overgedetermineerd stelsel** $Ac = f$ met A een N -bij- n -matrix. Dit stelsel kunnen we benaderend oplossen door $\|Ac - f\|$ te minimaliseren (of $\|D(Ac - f)\|$). D is hierin een soort van ‘**gewichtsmatrix**’ met strikt positieve diagonaalelementen. Op die manier zien we dat deze norm overeenkomt met het **normaalstelsel** (6.3) dat we willen minimaliseren.

Overfitting = bij een te hoge benaderingsgraad gaat de benaderende functie de ruis op de meetgegevens meer volgen, waardoor de onderliggende functie juist slechter wordt benaderd.

We gebruiken aparte **training data** en **test data**. Het bepalen van de coëfficiënten gebeurt door de traingingsfout (zie formularium) te minimaliseren. We kunnen de graad van de benaderende functie dan laten toenemen zolang de **generalisatiefout** daalt. Als die terug begint te stijgen, spreken we van **overfitting**. Het hebben van een aparte test dataset kan als verspilling worden gezien, dus tegenwoordig werkt men ook vaak met andere methodes zoals ‘**leave-one-out**’.

De algemene motivatie achter regularisatie is dat we coëfficiënten met grote absolute waarde willen vermijden. Deze zijn nl. erg gevoelig aan kleine veranderingen in de meetgegevens en dus heel ‘specifiek’. We voeren een extra term in bij de foutformule met **hyperparameter** λ die grote coëfficiënten afstraft (zie formualrium). Bij **Tikhonovregularisatie** is die term $F(c) = \|\Gamma c\|_2^2$.

Dit heeft als resultaat dat we het normaalstelsel slechts lichtjes moeten aanpassen: er worden enkel nieuwe termen bij de diagonaalelementen opgeteld. De eerder beschouwde methodes voor het oplossen van het normaalstelsel gelden dus nog steeds! Zie p. 164 in cursus en formularium voor een afleiding hiervan.

Hoofdstuk 7: Regularisatietechnieken

Bespreek de begrippen overfitting en regularisatie.

- (a) Wat is overfitting? Hoe kan je vaststellen over je benadering lijdt aan overfitting?
- (b) Hoe kan je overfitting vermijden? Bespreek kort het verschil tussen Tikhonovregularisatie en LASSO.
- (c) Hoe wordt regularisatie typisch ingevoerd bij het trainen van diepe neurale netwerken?

Overfitting = bij een te hoge benaderingsgraad gaat de benaderende functie de ruis op de meetgegevens meer volgen, waardoor de onderliggende functie juist slechter wordt benaderd.

We gebruiken aparte **training data** en **test data** om overfitting te detecteren. Het bepalen van de coëfficiënten gebeurt door de trainingsfout (zie formularium) te minimaliseren. We kunnen de graad van de benaderende functie dan laten toenemen zolang de **generalisatiefout** daalt. Als die terug begint te stijgen, spreken we van **overfitting**. Het hebben van een aparte test dataset kan als verspilling worden gezien, dus tegenwoordig werkt men ook vaak met andere methodes zoals '**leave-one-out**'.

De algemene motivatie achter regularisatie is dat we coëfficiënten met grote absolute waarde willen vermijden. Deze zijn nl. erg gevoelig aan kleine veranderingen in de meetgegevens en dus heel 'specifiek'. We voeren een extra term in bij de foutformule met **hyperparameter** λ die grote coëfficiënten afstraft (zie formularium).

- **Tikhonovregularisatie:** $F(c) = \|\Gamma c\|_2^2$. Het normaalstelsel moet slechts lichtjes worden aangepast, wat betekent dat we de bestaande methodes nog steeds kunnen gebruiken om het KKB-probleem op te lossen. We spreken van **ridge regression** als $F(c) = \|\lambda c\|_2^2$
- **LASSO-regularisatie:** $F(c) = \lambda |c|$. Tikhonovregularisatie heeft eerder als resultaat dat coëfficiënten kleiner worden gemaakt, maar niet geheel op 0 worden gezet (door het kwadratische karakter). LASSO-regularisatie gebruikt de 1-norm, waardoor dit probleem minder opduikt. Het geeft wel aanleiding tot niet-lineaire stelsels, waar we alternatieve methodes voor optimalisatie nodig zullen hebben.

Bij **neurale netwerken** willen we geen grote gewichten bij neuronen hebben, want dit betekent dat ze heel gevoelig worden voor verschillende soorten input, waardoor ze minder betrouwbaar worden als er nieuwe gegevens worden gepresenteerd. Analoog met andere regularisatietechnieken voegen we een term toe aan de kostfunctie die grote gewichtscoefficiënten afstraft. Biases worden vaak niet opgenomen in de regularisatieterm. Dit leidt tot een nieuwe parameter λ die we moeten kiezen:

$$Kost = \dots + \frac{\lambda}{N} \sum_{l=2} \|W^{[l]}\|_2^2$$

Deze aanpassing vereist slechts een zeer kleine en goedkope wijziging in de optimalisatie-routines.

Hoofdstuk 8: Grafen en eigenwaarden

Beschouw een netwerk beschreven door een enkelvoudige graaf. Leid een uitdrukking af voor het aantal directe verbindingen vanuit de knopen, in termen van de verbindingsmatrix. Geef een definitie van de meest centrale knoop in het netwerk die zich niet beperkt tot directe verbindingen. Hoe kan men deze berekenen?

We gaan uit van een binaire verbindingsmatrix A , waar een element 1 is als twee knopen verbonden zijn, en 0 als dat niet zo is. Omdat de graaf ongericht is, is matrix A symmetrisch. Het aantal bogen vertrekkend uit een knoop i is een eerste indicatie van de centraliteit:

$$\sum_{j=1}^N A_{ij} = \sum_{j=1}^N A_{ij}^2 = \sum_{j=1}^N A_{ij}A_{ji} = (A^2)_{ii}$$

Deze formule telt eigenlijk het aantal lussen van lengte 2 (heen en terug) die vertrekken in de knoop i . Om lussen van lengte n te berekenen veralgemenen we de formule:

$$(A^n)_{ii}$$

De centraliteit van het netwerk willen we niet enkel laten afhangen van het aantal directe burens. Lussen van grotere lengte moeten ook meegerekend worden, maar we geven ze een kleiner gewicht:

$$\frac{1}{2!}A^2 + \frac{1}{3!}A^3 + \dots + \frac{1}{n!}A^n + \dots$$

Omdat de diagonaal van A enkel nullen bevat, en omdat de 'waarde' van centraliteit enkel relatief gezien belangrijk is (we kunnen dus overal +1 optellen), kunnen we ook e^A berekenen (zie formularium). Dit is equivalent aan:

$$e^A = X \begin{bmatrix} e^{\lambda_1} & & \\ & \ddots & \\ & & e^{\lambda_N} \end{bmatrix} X^{-1}$$

Het berekenen van alle eigenwaarden doen we a.d.h.v. het **impliciet verschoven QR-algoritme**.

Besprek graafpartitionering.

- (a) Hoe komen we tot de formule van dit probleem?
- (b) Hoe 'relaxeren' we dit probleem?
- (c) Toon aan dat de oplossing de eigenvector is van de op een na kleinste eigenwaarde.
- (d) Welk algoritme/methode gebruiken we voor het berekenen van dit probleem?

We willen een verzameling knopen K (even aantal knopen) met bogen B (waar 'dubbele' verbindingen vermeden zijn) opsplitsen in twee deelgrafen K_1 en K_2 met evenveel knopen door zo weinig mogelijk bogen door te knippen.

We gebruiken de **graaf-Laplaciaan** L om de graaf te 'beschrijven' (zie formularium voor definitie). De partitie zelf stellen we voor door een vector x waarvoor geldt dat $x_i \in \{-1, 1\}$, afhankelijk of een knoop al dan niet in een bepaalde deelgraaf is terecht gekomen.

De voorwaarde dat de deelgrafen evengroot zijn kunnen we uitdrukken als:

$$[1 \dots 1]^T x = 0 \text{ of } \sum x = 0$$

Als we B_x de verzameling aan alle bogen tussen knopen van de twee deelgrafen noemen, kunnen we uitrekenen dat (de tussenstappen zijn redelijk complex, vraag eens aan ChatGPT):

$$\frac{1}{4} x^T L x = \frac{1}{4} \sum_{i=1}^N \sum_{j=1}^N L_{ij} x_i x_j = \frac{1}{2} \sum_{(i,j) \in B} (-x_i x_j + x_i^2) = \#(B_x)$$

Deze term willen we minimaliseren (zo weinig mogelijk bogen doorknippen).

De **binaire beperking** maakt het heel moeilijk om het probleem op te lossen. Een **brute-force** aanpak schaalgt gigantisch met N en is dus niet mogelijk. We versoepelen de eis:

$$x \in \{-1, 1\} \rightarrow \|x\|_2^2 = N$$

Vector x kan nu dus eender welk reëel getal aannemen.

L is symmetrisch en kan dus gediagonaliseerd worden: $L = V \Lambda V^T$ met V orthogonaal. Eigenwaarde 0 heeft eigenvector $[1 \dots 1]^T$ (stelling 8.3.3). Maken we nu substitutie $x = Vy$ dan geldt:

$$[1 \dots 1]^T x = 0 = v_N^T x = v_N^T V y = [0 \dots 0 \ 1] y = y_N$$

Het minimum probleem wordt dus omgezet:

$$\min \frac{1}{4} x^T L x = \min \frac{1}{4} y^T V^T V \Lambda V^T V y = \min \frac{1}{4} \sum \lambda_i y_i^2$$

Een orthogonale transformatie verandert de norm niet, de overige twee condities worden dan:

$$\|y\|^2 = N; y_N = 0$$

Om dit te minimaliseren geven we al het gewicht aan de op een na kleinste eigenwaarde λ_{N-1} .

Voor het vinden van de tweede-kleinste eigenwaarde van een typisch **spaarze symmetrische matrix** L , kunnen we **Lanczos' algoritme** gebruiken.

Bespreek het PageRank probleem.

- (a) Wat zijn de componenten van de graaf in werkelijkheid? Hoe kennen we een 'rank' toe aan knopen van de graaf?
- (b) Welk benaderend eigenwaardeprobleem bekomen we?
- (c) Hoe kunnen we dit omvormen tot een volwaardig eigenwaardeprobleem? Hebben de aanpassingen grote invloed op het resultaat?
- (d) Welk algoritme/methode gebruiken we voor het berekenen van dit probleem?

Hoofdstuk 9: Eigenwaardenalgoritmes

Beschouw het algoritme van Arnoldi.

- (a) Voor welk soort problemen is dit algoritme geschikt? Waarom? Geef een voorbeeld in de context van grafen.
- (b) Leg in detail uit hoe een orthonormale basis van een Krylov-ruimte wordt opgebouwd.
- (c) Hoe worden benaderingen van eigenwaarden bekomen? Leg het verband uit met één of andere orthogonale projectie.
- (d) Zou het zinvol zijn om ook een variant proberen te ontwikkelen gebaseerd op het gewijzigd Gram-Schmidt (*modified Gram-Schmidt*) algoritme? Waarom wel/ niet?

De **methode van Arnoldi** wordt gebruikt om een deel van de eigenwaarden te berekenen van een grote vierkante matrix A . Het is gebaseerd op de **Krylovruimte**. Elke iteratie zal dus een bijkomende eigenwaarde kunnen benaderen, en de overige eigenwaarden telkens beter gaan benaderen. Eigenwaarden aan de rand van het spectrum worden eerder benaderd.

Bij **graafpartitionering** moeten we de eigenvector vinden die hoort bij de een na kleinste eigenwaarde λ_{N-1} . Aangezien de **graaf-Laplaciaan** L hier een symmetrische matrix is, is **Lanczos' algoritme** aangewezen, een variant van de methode van Arnoldi.

De basis wordt opgebouwd via het **Gram-Schmidt algoritme**. De Krylovruimte wordt elke iteratie uitgebreid, dus hebben we nood aan een soort van 'geneste' structuur die on-the-fly kan uitgebreid worden. Zie algoritme 6 op p. 188 in de cursus.

Zie §9.1.3.2 op p. 188 in de cursus. We willen het residu zo klein mogelijk maken. Door loodrecht te projecteren op de Krylovruimte vinden we de beste benadering. De loodrechte projector is (zie H2) gelijk aan $P_D = Q_k Q_k^*$. Hieruit vinden we dat de eigenwaarden (**Ritzwaarden**) van $Q_k^* A Q_k = H_k$ de eigenwaarden van A zullen benaderen.

Ik neem aan dat met 'modified Gram-Schmidt' wordt bedoeld dat we de gehele Krylovbasis reeds moeten kennen. De te orthogonaliseren vectoren moeten op voorhand vastliggen. Gram-Schmidt opereert nl. op een gekende matrix A , terwijl het algoritme van Arnoldi elke iteratie een nieuwe vector creëert vanuit de laatste basisvector uit de vorige iteratie. $v_j = A q_j$ is numeriek een veel betere kandidaat dan $v_j = A^j b$. Voor grote j is die laatste nl. een goede benadering van eigenvector x_1 (zie methode van de machten). In stap 6 van het algoritme zouden dus **gevaarlijk aftrekkingen** gebeuren.

Bespreek in detail het convergentie-gedrag van het algoritme van Arnoldi en hoe het eigenwaarden lokaliseert. Maak hierbij gebruik van de *interpretatie* van formule

$$(A - \sigma I)Q_k = Q_k(H_k - \sigma I) + q_{k+1}\mathbf{e}_k^T h_{k+1,k}$$

en eigenschap

$$\hat{p} = \arg \min_{p \in M_k} \|p(A)b\|_2 \Leftrightarrow \hat{p}(z) = zI - H_k.$$

Wiskundige bewijzen zijn niet nodig.

De convergentie van de individuele **Ritz-waarden** naar de eigenwaarden verloopt kwalitatief **linear** (fout neemt met constante factor af). Uiteraard niet in de strikte zin, want vanaf m iteraties valt de Krylovruimte samen met $\mathbb{C}^m$.

De ‘extreme’ eigenwaarden worden als eerste teruggevonden, nl. de eigenwaarden aan de rand van het spectrum, maar dus niet per se de eigenwaarden met de grootste absolute waarde.

De eerste eigenschap is de **invariantie voor verschuivingen**. Het zegt dat als we de Arnoldi-iteratie toepassen op $(A - \sigma I, b)$ i.p.v. (A, b) , dat we dan Ritz-waarden $\{r_1^{(k)} - \sigma, \dots, r_k^{(k)} - \sigma\}$ krijgen.

Als voorbeeld nemen we een reële symmetrische matrix met eigenwaarden tussen $\lambda_{\min}$ en $\lambda_{\max}$. Intuïtief verwachten we dat Arnoldi de grootste eigenwaarde als eerste benaderd. Voeren we Arnoldi echter uit met $\sigma = \lambda_{\max}$, dan is nu $\lambda_{\min}$ juist de eigenwaarde aan de rand. De methode van Arnoldi differentieert dus niet tussen $\lambda_{\min}$ en $\lambda_{\max}$, in tegenstelling tot de deelruimte-iteratie.

De tweede eigenschap zegt dat als de Arnoldi-iteratie niet afbreekt, dat de karakteristieke veelterm van H_k (die kan gebruikt worden om eigenwaarde te berekenen) de unieke minimizer is van dat probleem, met M_k de verzameling van monische veeltermen van graad k .

We krijgen dan:

$$\min_{p \in M_k} \left\| X \begin{bmatrix} p(\lambda_1) & & \\ & \ddots & \\ & & p(\lambda_m) \end{bmatrix} X^{-1} \right\|$$

Intuïtief wordt duidelijk dat dit minimaal wordt, als er een monische veelterm bestaat die de eigenwaarden goed kan afdekken. Dit betekent dan dat de nulpunten van veelterm p de eigenwaarden benaderen.

Voorbeeld: als een matrix een reeks geclusterde eigenwaarden heeft en dan twee outliers, dan is het best om een veelterm te nemen die als nulpunt de twee outliers heeft en de resterende nulpunten via Chebyshevknoppunten verdeelt over de geclusterde eigenwaarde. Op die manier wordt de fout minimaal gehouden.

Bespreek hoe de deelruimte-iteratie de methode van de machten voor het berekenen van eigenwaarden veralgemeent.

- (a) Wat is de basisiteratie van de methode van de machten? Toon aan dat deze convergeert naar de dominante eigenvector. Wat is de convergentiefactor?
- (b) Hoe kunnen we de bijhorende dominante eigenwaarde schatten?
- (c) Hoe kunnen we de eigenwaarde met de kleinste modulus bekomen? Hoe kunnen we de eigenwaarde die het dichtst bij σ ligt benaderen?
- (d) Bespreek de deelruimte-iteratie en toon aan hoe die de n dominante eigenvectoren omsluit. Wat is de convergentiefactor?
- (e) Hoe kunnen we nu de eigenwaarden en -vectoren verkrijgen? Welke methode kunnen we hiervoor gebruiken?

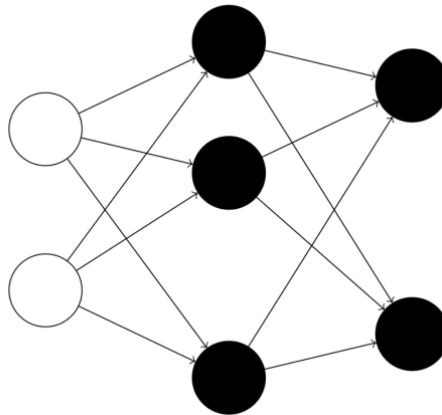
Bespreek het impliciet verschoven QR-algoritme.

- (a) Bespreek de deelruimte-iteratie op $p(A)$ i.p.v. A met $p(A) = (A - \rho_1 I) \dots (A - \rho_\ell I)$. Welke interpretatie kunnen we geven aan ρ_i ?
- (b) Waarom is dit algoritme inefficiënt?
- (c) Bespreek hoe impliciet QR-algoritme werkt voor een matrix $A^{(0)}$ in Hessenbergvorm.
Wat gebeurt er in één iteratiestap: $A^{(k)} = Q^{(k)*} A^{(k-1)} Q^{(k)}$?
- (d) Bewijs dat zo'n iteratiestap equivalent is met de deelruimte-iteratie op $p(A)$.
- (e) Hoe zou je ρ_i kiezen tijdens de iteratiestappen van het impliciet QR-algoritme voor (1) een symmetrische matrix, (2) een willekeurige matrix.

Hoofdstuk 10: Niet-lineaire benaderingsproblemen

Beschouw een diep neuraal netwerk.

- (a) Teken een neuraal netwerk van 3 lagen met 2 inputs, 3 neuronen in de middelste laag en 2 outputs.
- (b) Beschrijf in formulevorm de voorwaartse propagatie doorheen dit netwerk. Let erop de dimensies van alle objecten duidelijk te benoemen.
- (c) Met welke methode zou je dit neuraal netwerk trainen als je heel veel datapunten ter beschikking hebt? Leg uit welke hyperparameters (methodeparameters je nog moet kiezen).
- (d) Wat is backpropagation en waarom wordt het zo vaak gebruikt?
- (e) Waarom zouden we de stochastische versie van steilste helling gebruiken?



$$F(x) = \sigma(W^{[3]}\sigma(W^{[2]}x + b^{[2]}) + b^{[3]})$$

Hierin zijn matrices W de gewichten:

- $W^{[2]}$ is een 3-bij-2-matrix (2 input neuronen, 3 neuronen op middelste laag)
- $W^{[3]}$ is een 2-bij-3-matrix

Vectoren b zijn telkens een bias die we toevoegen nadat we de gewichten hebben toegepast aan de output van de vorige laag: $b^{[2]}$ heeft 3 elementen en $b^{[3]}$ heeft 2 elementen.

In totaal hebben we dus 17 parameters om te bepalen in dit neuraal netwerk.

‘Trainen’ van een netwerk = kiezen van de gewichten en biases op een manier die de **kostfunctie** minimaliseert (zie formularium voor kostfunctie). We zagen al een iteratief algoritme (steilste helling) dat elke stap de gewichts- en biasvector c aanpast in de richting met de steilste afdaling, nl. die van de gradiënt. Zo’n iteratiestap kan dan beschreven worden als:

$$c^{(k+1)} = c^{(k)} - \eta \nabla \text{Cost}(c^{(k)})$$

Hierin is η de **learning rate**, een maat die bepaalt hoe snel het algoritme de gewichten gaat aanpassen naar nieuwe waarden. We kiezen initiële gewichten en itereren totdat een stopcriterium of het maximum aantal iteraties is bereikt.

Hierbij is het berekenen van gradiënten van groot belang. Een efficiënte manier om dit te doen is via **backpropagation**. Backpropagation is equivalent met de kettingregel voor afgeleiden. We doorlopen een gelineariseerde versie van het neurale netwerk achterwaarts. De efficiënte berekening van de gradiënt van de kostfunctie maakt de **methode van de steilste helling** erg interessant om neurale netwerken te trainen.

Als we veel neuronen hebben is het berekenen van de gradiëntvector bij elke iteratie heel rekenintensief. We kiezen daarvoor om met een **stochastische gradiëntmethode** te werken.

- We kiezen een willekeurig datapunt uit te verzameling en we berekenen de gradiënt zo dat die enkel de kost op dat datapunt minimaliseert (zie twee laatste formules in formularium).
- Hier kunnen we via een steekproef met of zonder teruglegging werken.
- Zelfs met een kleine leersnelheid garandeert zo'n iteratie niet altijd dat de algemene kostfunctie vermindert, omdat we slechts één monster gebruiken. We kunnen dus ook werken met **mini-batches** van monsters.

Daarnaast kunnen we de leersnelheid en andere hyperparameters dynamisch doen variëren naarmate het trainingsproces vordert. Op die manier wordt het rekenwerk enorm vermindert en kunnen we onze neurale netwerken schalen tot miljoenen/miljarden neuronen.

Een laatste opmerking gaat over **regularisatie**. We willen geen grote gewichten bij neuronen hebben, want dit betekent dat ze heel gevoelig worden voor verschillende soorten input. Analoog met andere regularisatietechnieken voegen we een term toe aan de kostfunctie die grote gewichtscoëfficiënten afstraft. Biases worden vaak niet opgenomen in de regularisatieterm. Dit leidt tot een nieuwe parameter die we moeten kiezen:

$$Kost = \dots + \frac{\lambda}{N} \sum_{l=2} \|W^{[l]}\|_2^2$$

Hoofdstuk 11: Optimalisatie-algoritmes

Beschouw de methode van de toegevoegde gradiënten voor een lineair stelsel.

- (a) Schrijf het algoritme in pseudocode en leg elk van de stappen uit.
- (b) Toon aan dat de methode een Krylovruimte opbouwt.
- (c) Toon aan dat de methode in elke iteratie de beste benadering vindt van de oplossing in de bijhorende Krylovruimte.
- (d) Kan je een verband geven met veeltermbenadering? Hoe kan dit verband uitgebuit worden op de methode van de toegevoegde gradiënten te versnellen?

Algoritme van toegevoegde gradiënten:

Algoritme 10 Algoritme van de toegevoegde gradiënten (basis)

Require: $x^{(0)}$

```
1:  $p^{(0)} = -\nabla f(x^{(0)})$ 
2: for  $k = 1, 2, \dots$  do
3:    $\alpha^{(k)} = \arg \min_{\alpha} f(x^{(k-1)} + \alpha p^{(k-1)})$ 
4:    $x^{(k)} = x^{(k-1)} + \alpha^{(k)} p^{(k-1)}$ 
5:    $\beta^{(k)} = \frac{\|\nabla f(x^{(k)})\|^2}{\|\nabla f(x^{(k-1)})\|^2}$ 
6:    $p^{(k)} = -\nabla f(x^{(k)}) + \beta^{(k)} p^{(k-1)}$ 
7: end for
```

Doel: stelsel $Ax = b$ oplossen.

Methode: doelfunctie minimaliseren. Uit stelling 11.1.1 weten we dat dit gebeurt als:

$$\nabla f(x) = 0 \text{ dus } Ax - b = 0 \rightarrow \nabla f(x) = Ax - b$$

- Lijn 1: beginrichting kiezen. Negatieve gradiënt is beste keuze.
- Lijn 3: zoek de beste stapgrootte $\alpha^{(k)}$. Daar waar de gradiënt loodrecht staat op de zoekrichting vinden we de beste richting. Dit betekent dat het scalair product 0 moet zijn, wat een uitdrukking voor $\alpha^{(k)}$ geeft. Dit kunnen we nog omzetten naar de $\alpha^{(k)}$ die in het formularium staat.
- Lijn 4: update x naar nieuwe richting. In het algoritme kunnen we ook het residu $r^{(k)} = b - Ax^{(k)}$ telkens updaten, omdat we dit nodig gaan hebben tijdens berekeningen. De negatieve gradiënt is dan gelijk aan het residu, dus gebruiken we het residu als negatieve gradiënt.
- Lijn 5-6: de nieuwe zoekrichting is nu een combinatie van de optimale zoekrichting en de vorige zoekrichting. De factor $\beta^{(k)}$ is gekozen om snellere convergentie te hebben.

Algoritme 12 Algoritme van de toegevoegde gradiënten voor stelsels (standaard vorm)

Require: $A > 0$, b , $k_{\max}$, $\epsilon > 0$

```
1:  $p^{(0)} = r^{(0)} = B$ ,  $x^{(0)} = 0$ 
2: for  $k = 1, 2, \dots, k_{\max}$  do
3:    $\alpha^{(k)} = \frac{r^{(k-1)T} r^{(k-1)}}{p^{(k-1)T} A p^{(k-1)}}$ 
4:    $x^{(k)} = x^{(k-1)} + \alpha^{(k)} p^{(k-1)}$ 
5:    $r^{(k)} = r^{(k-1)} - \alpha^{(k)} A p^{(k-1)}$ 
6:   if  $\|r^{(k)}\|_2 < \epsilon$  then
7:     return  $x^{(k)}$ 
8:   end if
9:    $\beta^{(k)} = \frac{r^{(k)T} r^{(k)}}{r^{(k-1)T} r^{(k-1)}}$ 
10:   $p^{(k)} = r^{(k)} + \beta^{(k)} p^{(k-1)}$ 
11: end for
```

Bewijzen dat het algoritme een Krylovruimte opbouwt kan men intuïtief doen:

- Elk nieuw residu wordt opgebouwd uit het vorig residu en een toepassing van A . Het nieuw residu ligt dus in een grotere Krylovruimte.
- Elke nieuwe richting wordt opgebouwd uit het nieuwe residu en de vorige richting.
- Elke nieuwe oplossing x wordt opgebouwd uit de vorige oplossing en de nieuwe richting.
- In symbolen leidt het volgende tot eigenschap 11.2:

$$\begin{aligned} r_k &= b - A x_k \in \mathcal{K}_k(A, r_0) \\ p_k &= r_k + \beta_{k-1} p_{k-1} \in \mathcal{K}_k(A, r_0) \\ x_{k+1} &= x_k + \alpha_k p_k \in x_0 + \mathcal{K}_{k+1}(A, r_0) \end{aligned}$$

Om te bewijzen dat elke iteratie de beste oplossing vindt in de bijhorende Kryloveruimte, moeten we eerst (11.7) en (11.8) bewijzen (staan ook in formularium). (Op examen best even vragen of dit nodig is!)

Dit doen we met inductie. Dat beide eigenschappen gelden voor $k = 1$ kan je makkelijk nagaan. Voor de inductiestap doen we het volgende:

- Schrijf lijn 5 uit (residu berekenen) en vermenigvuldig langs links
- Voor $j < k - 1$ mogen we $r^{(j)}$ vervangen door $p^{(j)}$ omdat ze beiden tot dezelfde Krylovruimte behoren (?). De uitdrukking wordt dan 0, dit deel is dus al bewezen.
- Voor $j = k - 1$ wordt het rechterlid 0 op voorwaarde dat $\alpha^{(k)} = \dots$ (herschrijven)
- Door lijn 10 in te vullen, de tweede term in het rechterlid te schrappen (want 0) en langs links te vermenigvuldigen, bekomen we de $\alpha^{(k)}$ die we eerder besproken hebben.

Voor de andere eigenschap schrijf je gewoon op het examen dat die analoog wordt bewezen, nu gebruik makende van lijn 10.

De uitdrukking voor de minimale fout staat in het formularium en wordt bewezen op p. 227 in het handboek. Belangrijkste is gewoon om $x = x^{(k)} + \Delta x$ in te voeren en dan de uitdrukking uit te werken. De fout is dan ‘minimaal t.o.v. de A -norm’.

Ook het verband met de veeltermbenadering staat in het formularium. Concreet betekent dit dat de grootte van de fout na k iteraties voor een groot deel wordt bepaald door hoe goed het spectrum van A 'afgedekt' kan worden door een veelterm. Concreet is het voor een snelle convergentie bevorderlijk als de eigenwaarden van A gegroepeerd zijn in een klein aantal clusters.

Men kan ook een **preconditioner** toevoegen om de eigenwaarden beter te clusteren. Als het matrixproduct hieronder gemakkelijk kan berekend worden, dan kan dit de convergentie versnellen.

$$(D^{-1}AD^{-T})y = D^{-1}b$$

De eigenwaarden van $(D^{-1}AD^{-T})$ zijn in het algemeen verschillende van A en als deze beter geclusterd zijn, is dit beter voor de convergentie.

Beschouw een niet-lineair optimalisatieprobleem.

- Toon aan dat de doelfunctie het sterkste daalt in de richting van de negatieve gradiënt van de doelfunctie.
- Om de stapgrootte te bepalen die we zetten in die richting hebben we twee manieren gezien: een lijnzoekmethode en backtracking met de Armijovoorwaarde. Leg van beide methodes de werking uit, en vergelijk hun eigenschappen.

We kunnen de afgeleide van $f(x)$ berekenen in een bepaalde richting. Zo'n richting zullen we dit meestal aangeven als eenheidsvector s . Nu zien we dat $x(\alpha) = x^* + \alpha s$ een rechte lijn beschrijft volgens richting s . Op die manier wordt functie $F(x(\alpha))$ slechts een functie in één scalaire veranderlijke. De afgeleide van f in richting s is dan:

$$\frac{dF(\alpha)}{d\alpha} = (\nabla f(x))^T s$$

Uit de **ongelijkheid van Cauchy-Schwartz**, zie (3.33), volgt:

$$(\nabla f(x))^T s \leq -\|\nabla f(x)\| \cdot \|s\|$$

De gelijkheid treedt op als $s = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$. De afgeleide is dan het kleinst.

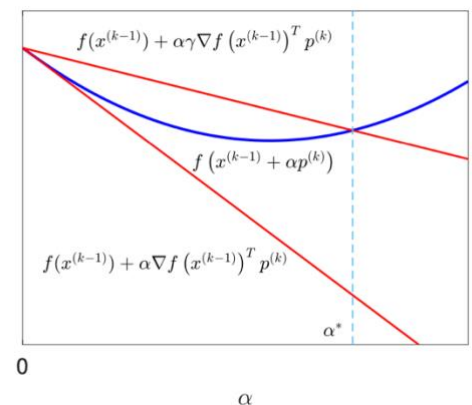
De iteratieve methode van de steilste afdaling bestaat uit twee stappen:

- Bereken de zoekrichting. Hiervoor zagen we al dat we hiervoor $-\nabla f(x)$ voor nemen.
- Zoek de stapgrootte zodat de nieuwe benadering een voldoende lagere functiewaarde geeft dan de vorige stap.

De **exakte lijnzoekmethode** gaat voor elke iteratiestap de optimale α vinden die de kleinste functiewaarde geeft (zie formularium). Dit geeft echter niet altijd een snelle convergentie. Daarenboven moeten we een scalair optimalisatieprobleem oplossen, wat ook vaak iteratief moet gebeuren.

In praktijk worden daarom vaak **backtracking** methodes toegepast. We vertrekken van $\alpha = 1$ en verlagen die totdat de nieuwe functiewaarde aan een voorwaarde voldoet. De **Armijovoorwaarde** staat in het formularium. γ is een parameter van het algoritme.

- Voorkomt te kleine stappen. Op de figuur zien we dat als we gewoon de raaklijn nemen (onderste rode lijn), we een kleine stap bekomen. Kleine stappen hebben het risico om vroegtijdig stil te vallen voordat een minimum bereikt is.
- De helling van de raaklijn wordt met een factor vermindert. We vermijden ook te grote stappen, die tot instabiliteit kunnen leiden.
- Garandeert een monotone daling van de functiewaarde per iteratie
- Op het einde is precisie vereist: ook dit wordt gegarandeerd. De Armijo-voorwaarde is dus een goede balans tussen te kleine en te grote stappen.



Hoofdstuk 12: lje representaties en benaderingen

Bespreek in detail twee methodes om een goede lage-rangbenadering van een matrix te bekomen. Vergelijk deze. Sta daarbij ook stil bij de hoeveelheid rekenwerk en de kwaliteit van de benadering.

QR-factorisatie met kolomwisselingen:

- Idee: na de QR-factorisatie, gebruik de eerste k kolommen van Q en de overeenkomstige rijen van R .
- Doel: belangrijke informatie blijft behouden. In elke stap voegen we dus een extra check toe die de grootste kolom naar voren brengt door, indien nodig, een kolomwisseling uit te voeren. Hierdoor ontstaat er een extra permutatiematrix: $AP = QR$.
- Intuïtief: $|r_{ii}| \geq |r_{ij}|$ met $i \leq j$, dus de eerste rijen van R wegen meer door in de factorisatie dan de andere, in de zin dat ze een belangrijkere bijdrage tot A leveren.
- Rekenwerk: cfr. QR-factorisatie, met extra check waar we de norm van elke kolom moeten berekenen.

Singuliere waarden ontbinding (SWO)

- Idee: na SWO-ontbinding, gebruik de eerste k kolommen van U , k -bij- k -matrix van Σ en k rijen van V^* .
- SWO-benadering is volgens de spectrale norm én de Frobeniusnorm de beste benadering van A (stelling 12.2.1).
- Rekenwerk: Het berekenen van de SWO wordt gedaan a.d.h.v. de eigenwaardeontbinding van $H = \begin{bmatrix} 0 & A_0^* \\ A_0 & 0 \end{bmatrix}$, omdat dit stabiel is dan de eigenwaardeontbinding van A^*A .

Die A_0 bekomen we door de matrix A om te vormen tot een bi-diagonale matrix. Het is namelijk gemakkelijker om de eigenwaarden hiervan te berekenen. Eigenwaardeontbinding is een inherent iteratief proces, wat dus meer rekenwerk kost in het algemeen.

- Hoe zou je $\|A\|_2$ berekenen? Hoe zou je $\|A\|_F$ berekenen?
- Hoe wordt PCA toegepast voor datacompressie?

De spectrale norm A is gelijk aan de grootste singuliere waarde. Om die te berekenen moeten we een eigenwaardeprobleem oplossen.

- We vormen de matrix om tot een bi-diagonale matrix via unitaire transformaties. Hieruit halen we een vierkante matrix A_0 . De linker- en rechtermatrices kunnen onafhankelijk gekozen worden (dit is niet zo in bij impliciet verschoven QR-algoritme, omdat we daar gelijkvormigheid moeten vrijwaren?)
- Een algoritme waar we de eigenwaarden van $A^*A = V\Sigma^*\Sigma V^T$ berekenen is numeriek niet stabiel.
- We berekenen de eigenwaarden van $H = \begin{bmatrix} 0 & A_0^* \\ A_0 & 0 \end{bmatrix}$ (je kan nagaan dat dit ook een eigenwaardeprobleem is, zie formularium).
- De methode van de machten convergeert naar de dominante eigenwaarde. Echter, H berekent zowel de positieve als negatieve eigenwaarden van H (= singuliere waarden van A), dus moeten we deelruimte iteratie hanteren met $n=2$.

Voor de Frobeniusnorm hebben we alle singuliere waarden nodig, dus hanteren we het impliciet verschoven QR-algoritme.

Principal component analysis (PCA) heeft als doel de dimensies van een dataset te reduceren door de belangrijkste richtingen over te houden.

Voorwaarden: principal components staan loodrecht op elkaar en zijn genormeerd, zie formularium. Als we de SWO hierin substitueren met $r = Vy$ zien we gemakkelijk dat $r_i = v_i$. Intuïtief: de grootste singuliere waarden geven meer doorslag in de SWO-ontbinding van een vector x (zie tweede formule in formularium).

Op die manier kunnen we het aantal variabelen in een dataset reduceren en enkel de belangrijkste 'richtingen'/eigenschappen van de dataset verder gebruiken. Dit is vaak verantwoord omdat fysische systemen en processen vaak inherent laag-dimensionaal zijn.